# Comparison to Existing Tools

## Comparison to Existing Tools I

The gap analysis established that no single tool integrates all six cross-cutting concerns. Here we synthesize the fourteen-dimension comparison into three structural insights. First, the landscape bifurcates: workflow managers (Snakemake (Köster and Rahmann 2012), Nextflow (Di Tommaso et al. 2017), CWL (Amstutz et al. 2016)) provide distributed execution but no manuscript support; publication tools (Quarto (Allaire et al. 2024), Jupyter Book, R Markdown (Xie et al. 2018), Overleaf (Overleaf (Digital Science) 2025), Prism (OpenAI 2026)) author documents but embed no integrity guarantees; and DVC (Iterative, Inc. 2024) versions artifacts without orchestrating pipelines. `template/` occupies the intersection, sacrificing distributed execution for unified enforcement of testing, provenance, and documentation. This positioning is complementary—a mature deployment might use Nextflow upstream and `template/` for rendering, testing enforcement, and provenance downstream. Typst (Mädje and Haug 2023), with its faster compilation cycle, is not one of the nine compared peers but could serve as an alternative rendering backend if a Pandoc writer were contributed.

Second, the eight enforcement capabilities `template/` co-locates—testing enforcement, coverage thresholds, steganographic watermarking, multi-project management, AI-agent documentation, the agentic skill protocol, an interactive TUI, and Zero-Mock policy—are individually straightforward (and several, such as multi-project management and AI-agent documentation, are matched in part by individual peers). Their value here lies in a shared pipeline contract: required test and documentation gates run together, while selected provenance stages carry their own validation. The FAIR4RS principles (Barker et al. 2022; Lamprecht et al. 2020) articulate what research software quality

## Comparison to Existing Tools I

requires; FAIRsoft (Garijo et al. 2024) scores compliance observationally; `template/` operationalizes part of this agenda by failing project tests below the declared 90% floor and validating provenance artifacts when those stages are enabled. Cohen et al.'s four pillars of research software engineering (Cohen et al. 2021)—sustainability, quality, community, and policy—are addressed primarily through the first two pillars via quality-gated automation.

## Comparison to Existing Tools I

Third, the AI-agent documentation dimension reveals an underserved need. Overleaf and Prism provide AI *writing* assistance, but neither exposes structured documentation for *external* agents to consume. `template/`'s `AGENTS.md` + `SKILL.md` layer enables an agent entering the repository to discover capabilities, understand API contracts, and invoke modules without prior training (Documentation Duality, AI Collaboration).

# FAIR4RS Evolution (2024–2026) I

Since the FAIR4RS principles were published (Barker et al. 2022), the community has moved toward operationalization. The RDA Virtual Plenary 24 (April 2025) featured a two-year retrospective review (Honeyman et al. 2024) recommending principle amendments—notably adding *reproducibility* as an explicit requirement and clarifying "domain-relevant standards"—alongside a leadership refresh and parallel guidance activities. The ReSA Actionable FAIR4RS Task Force (launched December 2024) analyzed the 17 principles into six actionable categories (identifiers, metadata for publication/discovery/reuse, standards, references, and licenses), with a first draft expected by September 2025 (Research Software Alliance 2024). Tools for automated FAIR assessment have also matured: the F-UJI extension for research software evaluation now scores against FRSM-04 through FRSM-17 metrics, complementing Garijo et al.'s FAIRsoft evaluator (Garijo et al. 2024). `template/`'s pipeline-enforced quality gates—coverage thresholds, documentation completeness checks, and provenance embedding—anticipate this operationalization trend by implementing FAIR4RS not as a post-hoc assessment but as an architectural invariant.

In Gentleman and Temple Lang's terminology (Gentleman and Temple Lang 2007), `template/` is a *research compendium* scaled to the repository level—bundling not just one study's code and data but N studies, with shared infrastructure, automated testing, and embedded provenance. Nüst et al.'s executable research compendium (ERC) (Nüst et al. 2017) extends this vision with containerized reproduction environments; `template/` complements containerization by adding the testing enforcement, multi-project management, and provenance embedding layers that ERCs do not address.